

ASP.NET Core API Versioning

Comprehensive Guide to Principles, Strategies, and Implementation

1. What is API Versioning and Why is it Needed?

What is API Versioning?

API Versioning is the practice of managing changes to an API's contract over time. It allows developers to safely introduce structural changes to request/response payloads, URL structures, or behavioral logic without breaking existing client integrations. Clients explicitly specify which version of the API they are designed to interact with.

Why is it Needed?

Once an API is published and consumed by clients (mobile apps, single-page applications, external partners), the API contract is essentially immutable for those clients. Modifying the contract introduces breaking changes.

- **Contract Stability:** Clients depend on a specific schema. If a field name changes from `firstName` to `given_name`, parsing on the client side will fail.
- **Backward Compatibility:** Versioning ensures that older clients continue to function normally while new clients can leverage new features.
- **Lifecycle Management:** It provides a clear mechanism to deprecate and eventually sunset older API contracts.

2. Principles of API Versioning

- **Version only when necessary:** Additive changes (e.g., adding a new, non-required field to a response) are generally non-breaking and do not require a new major version.
- **Breaking changes dictate new versions:** Removing a field, changing a data type, changing URL paths, or modifying business rules that affect state transitions warrant a version bump.
- **Postel's Law (Robustness Principle):** APIs should be conservative in what they send and liberal in what they accept. Clients should ignore unrecognized fields, which helps avoid unnecessary version bumps for additive changes.

3. Types of Versioning Strategies

There is no universally "correct" versioning strategy. The choice depends on the specific requirements of the consumers and the architectural style.

Strategy 1: URI Path Versioning

The version is embedded directly into the URL path.

Example: `GET /api/v1/vehicles`

Advantages	Trade-offs
- Extremely explicit; developers can see the version immediately. - Easily cacheable at the network edge. - Easy to test in a browser without custom tools.	- Violates strict REST principles (a URI should identify a resource, not a version). - If the underlying resource hasn't changed between versions, you duplicate endpoints unnecessarily.

Strategy 2: Query String Versioning

The version is appended as a query parameter.

Example: `GET /api/vehicles?api-version=1.0`

Advantages	Trade-offs
- Keeps the resource URI clean and REST-compliant. - Easy to specify a default version if the query parameter is omitted.	- Can interfere with other query parameters like filtering or pagination. - Slightly less intuitive to read in logs compared to path versioning.

Strategy 3: Custom Header Versioning

The client sends the version inside an HTTP request header.

Example: `X-API-Version: 2.0`

Advantages	Trade-offs
- Strictly adheres to REST principles; URIs remain completely unchanged. - Does not clutter the URL.	- Harder to test manually; requires tools like Postman or cURL. - Can cause issues with caching mechanisms (Requires configuring the <code>Vary: X-API-Version</code> response header).

Strategy 4: Media Type (Accept Header) Versioning

Also known as Content Negotiation versioning. The client modifies the `Accept` header.

Example: `Accept: application/vnd.myapi.v2+json`

Advantages	Trade-offs
- Architecturally the most RESTful approach. - Allows versioning of specific representations rather than the entire API endpoint.	- High complexity for consumers to implement. - Difficult to test via browser. - Heavy reliance on proper Content Negotiation implementation.

4. Implementation in Modern ASP.NET Core

For modern ASP.NET Core (.NET 6/7/8+), the standard library for versioning is maintained by the .NET Foundation. *Note: The older `Microsoft.AspNetCore.Mvc.Versioning` is deprecated.*

Step 1: Install NuGet Packages

```
dotnet add package Asp.Versioning.Http
dotnet add package Asp.Versioning.Mvc.ApiExplorer
```

Step 2: Configure Dependency Injection (Program.cs)

You can configure multiple versioning strategies simultaneously (e.g., allowing clients to use either the Query String or Header).

```

using Asp.Versioning;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();

// Configure API Versioning
builder.Services.AddApiVersioning(options =>
{
    // Return a specific header to the client indicating supported versions
    options.ReportApiVersions = true;

    // If client doesn't specify a version, assume default
    options.AssumeDefaultVersionWhenUnspecified = true;

    // Set the default version to 1.0
    options.DefaultApiVersion = new ApiVersion(1, 0);

    // Define how the API reads the version from the request
    options.ApiVersionReader = ApiVersionReader.Combine(
        new UriSegmentApiVersionReader(),
        new HeaderApiVersionReader("x-api-version"),
        new QueryStringApiVersionReader("api-version")
    );
});

.AddMvc() // Required for Controllers
.AddApiExplorer(options =>
{
    // Configure Swagger integration format: 'v' + major + minor
    options.GroupNameFormat = "'v'VVV";
    options.SubstituteApiVersionInUrl = true;
});

var app = builder.Build();
app.MapControllers();
app.Run();

```

Technical Context: `ReportApiVersions = true` instructs the middleware to append headers like `api-supported-versions` and `api-deprecated-versions` to the HTTP response. This allows clients to dynamically discover lifecycle changes.

Step 3: Apply Versions to Controllers

Decorate controllers with the `[ApiVersion]` attribute. Use `[MapToApiVersion]` on specific actions if you have multiple versions handled within the same controller.

```

[ApiController]
// Include {version:apiVersion} in the route constraint for Path Versioning
[Route("api/v{version:apiVersion}/{controller}")]
[ApiVersion("1.0")] // Defines this controller handles v1.0
[ApiVersion("2.0")] // Also handles v2.0
public class VehiclesController : ControllerBase
{
    [HttpGet]
    [MapToApiVersion("1.0")] // Maps specifically to v1.0
    public IActionResult GetV1()
    {
        return Ok(new { Message = "Returning v1 Data: Minimal details." });
    }

    [HttpGet]
    [MapToApiVersion("2.0")] // Maps specifically to v2.0
    public IActionResult GetV2()
    {
        return Ok(new { Message = "Returning v2 Data: Extended details." });
    }
}

```

Design Consideration: Keeping multiple versions within the same controller file often leads to messy, difficult-to-maintain code as versions diverge. It is generally preferable to separate controllers into version-specific namespaces (e.g., `MyApi.Controllers.V1.VehiclesController` and `MyApi.Controllers.V2.VehiclesController`).

5. Deprecation and Sunsetting

When an API version is nearing the end of its lifecycle, you should formally deprecate it before removing it entirely.

```

[ApiVersion("1.0", Deprecated = true)]
[ApiController]
[Route("api/v{version:apiVersion}/{controller}")]
public class OldVehiclesController : ControllerBase
{
    // Implementation
}

```

Setting `Deprecated = true` will not disable the endpoint. Instead, when `ReportApiVersions = true` is configured, the response will include the header `api-deprecated-versions: 1.0`, signaling to consuming developers that they need to migrate.